Informe de Laboratorio: Estructura de Computadores

Nombre del Estudiante: Cindy Eliana Peña Villanueva - Victor Andrés Cubillos Baquero
Fecha: 2 de marzo de 2026
Asignatura: Estructura de Computadores

Enlace del repositorio en GitHub: https://github.com/Andrety8/mips-pipeline-analysis_Cindy_Victor.git

https://eliana-pe.github.io/Simulaci-n-Pipeline-MIPS-5-etapas-Cindy-Pena-y-Victor-Cubillos/

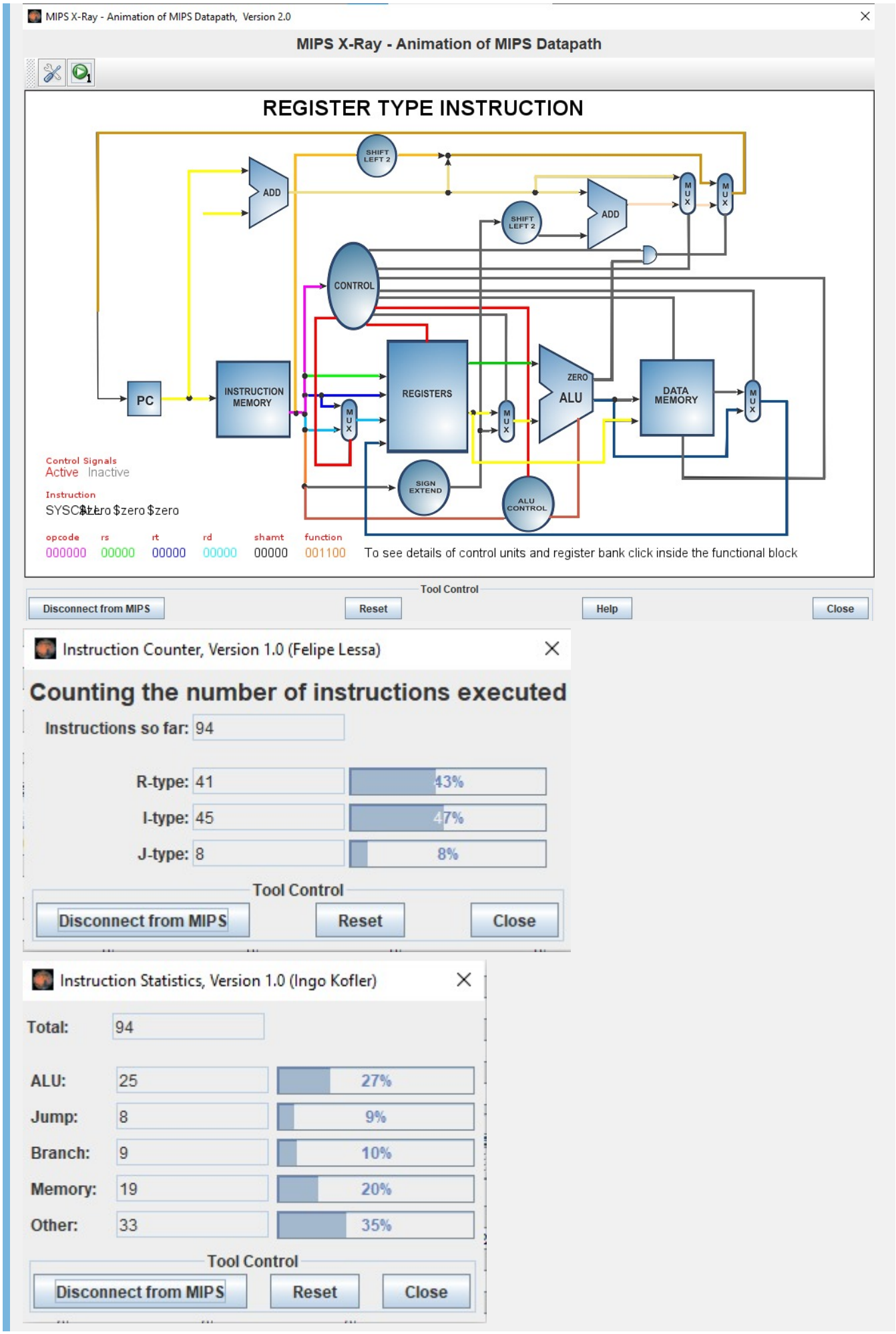
---

1. Análisis del Código Base

1.1. Evidencia de Ejecución

Adjunte aquí las capturas de pantalla de la ejecución del `programa_base.asm` utilizando las siguientes herramientas de MARS:

- **MIPS X-Ray** (Ventana con el Datapath animado).
- **Instruction Counter** (Contador de instrucciones totales).
- **Instruction Statistics** (Desglose por tipo de instrucción).


MIPS X-Ray - Animation of MIPS Datapath, Version 2.0
MIPS X-Ray - Animation of MIPS Datapath
REGISTER TYPE INSTRUCTION
SHIFT LEFT 2
ADD
SHIFT LEFT 2
ADD
MUX
MUX
CONTROL
PC
INSTRUCTION MEMORY
REGISTERS
ZERO
ALU
DATA MEMORY
MUX
MUX
MUX
SIGN EXTEND
ALU CONTROL
Control Signals
Active Inactive
Instruction
SYSCALL $zero $zero $zero
opcode rs rt rd shamt function
000000 00000 00000 00000 00000 001100
To see details of control units and register bank click inside the functional block
Tool Control
Disconnect from MIPS
Reset
Help
Close
Instruction Counter, Version 1.0 (Felipe Lessa)
Counting the number of instructions executed
Instructions so far: 94
R-type: 41 43%
I-type: 45 47%
J-type: 8 8%
Tool Control
Disconnect from MIPS
Reset
Close
Instruction Statistics, Version 1.0 (Ingo Kofler)
Total: 94
ALU: 25 27%
Jump: 8 9%
Branch: 9 10%
Memory: 19 20%
Other: 33 35%
Tool Control
Disconnect from MIPS
Reset
Close

1.2. Identificación de Riesgos (Hazards)

Completa la siguiente tabla identificando las instrucciones que causan paradas en el pipeline:

Instrucción Causante	Instrucción Afectada	Tipo de Riesgo	Ciclos de Parada
`lw $t6, 0($t5)`	`mul $t7, $t6, $t0`	RAW Load-Use (uso inmediato del dato cargado)	1 ciclo
`mul $t7, $t6, $t0`	`addu $t8, $t7, $t1`	RAW (dependencia de registro) con forwarding	0 ciclos
`addu $t8, $t7, $t1`	`sw $t8, 0($t9)`	RAW (store-data) con forwarding	0 ciclos

Análisis

> El único riesgo que genera una parada real en el pipeline es el **Load-Use** entre `lw` y `mul`, ya que el dato cargado desde memoria no está disponible inmediatamente para la siguiente instrucción.

> Las demás dependencias RAW se resuelven mediante **forwarding**, por lo que no introducen ciclos de espera adicionales.

> En consecuencia, el código base presenta **1 stall por iteración del bucle**.

1.2. Estadísticas y Análisis Teórico

Dado que MARS es un simulador funcional, el número de instrucciones ejecutadas será igual en ambas versiones. Sin embargo, en un procesador real, el tiempo de ejecución (ciclos) varía. Completa la siguiente tabla de análisis teórico:

Métrica	Código Base	Código Optimizado
Instrucciones Totales (según MARS)	94	94
Stalls (Paradas) por iteración	1	0
Total de Stalls (8 iteraciones)	8	0
Ciclos Totales Estimados (Inst + Stalls)	102	94
CPI Estimado (Ciclos / Inst)	1,085	1,00

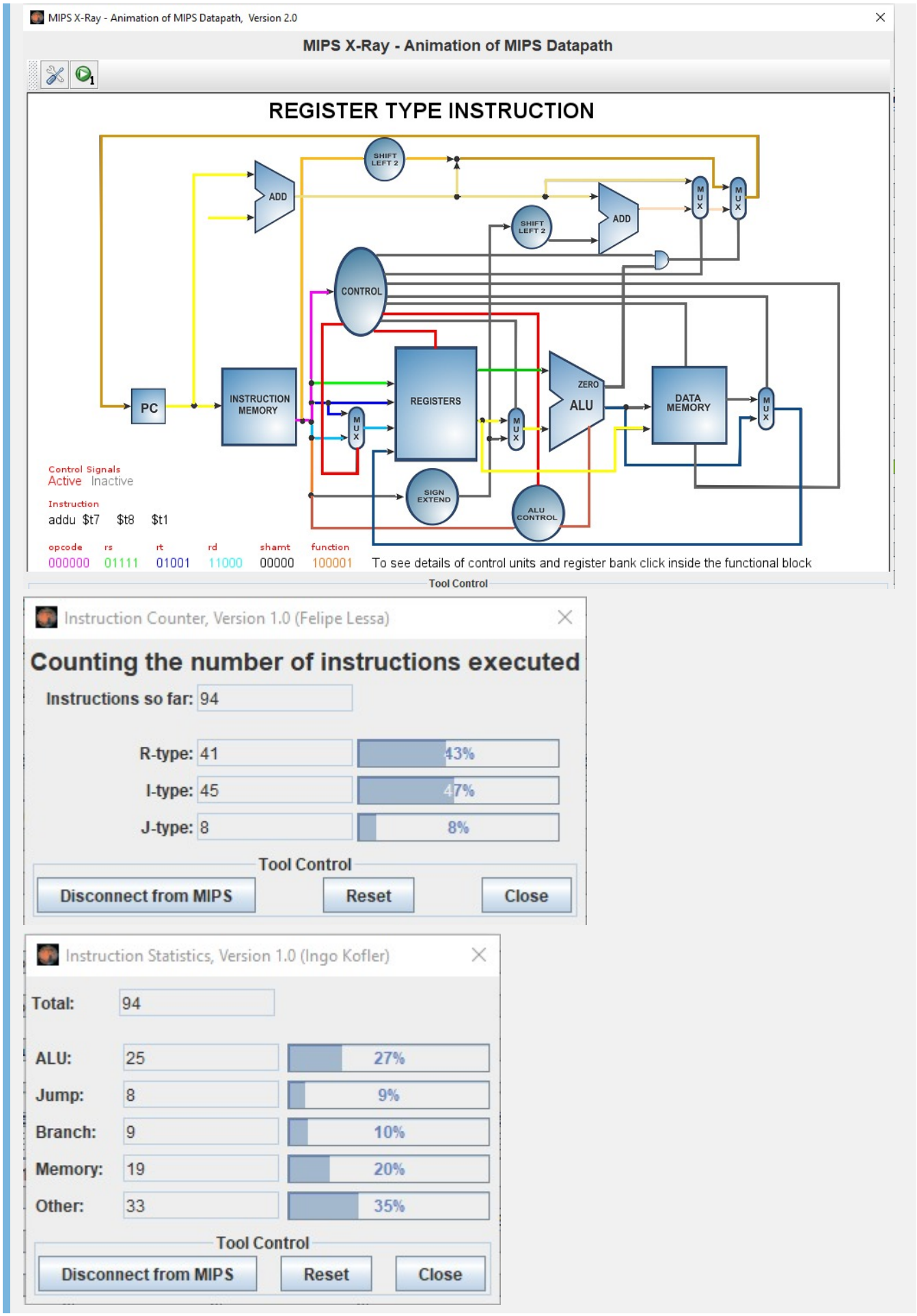
2. Optimización Propuesta

2.1. Evidencia de Ejecución (Código Optimizado)

Adjunte aquí las capturas de pantalla de la ejecución del `programa_optimizado.asm` utilizando las mismas herramientas que en el punto 1.1:

- **MIPS X-Ray**.
- **Instruction Counter**.

- **Instruction Statistics**.



2.2. Código Optimizado

Pega aquí el fragmento de tu bucle `loop` reordenado:

```
.data
vector_x: .word 1, 2, 3, 4, 5, 6, 7, 8
vector_y: .space 32              # 8 enteros (8 * 4 bytes)
const_a:  .word 3
const_b:  .word 5
tamano:   .word 8

.text
.globl main

main:
    # --- Inicialización ---
    la   $s0, vector_x          # Dirección base de X
    la   $s1, vector_y          # Dirección base de Y
    lw   $t0, const_a           # Cargar constante A
    lw   $t1, const_b           # Cargar constante B
    lw   $t2, tamano            # Cargar tamaño del vector (n)
    li   $t3, 0                 # i = 0

loop:
    # --- Condición de salida ---
    beq  $t3, $t2, fin          # Si i == n → terminar

    # --- Cálculo de offset (i * 4 bytes) ---
    sll  $t4, $t3, 2            # t4 = i * 4

    # --- Dirección de X[i] ---
    addu $t5, $s0, $t4          # t5 = &X[i]

    # --- Carga desde memoria ---
    lw   $t6, 0($t5)            # t6 = X[i]

    # --- Relleno del Load-Use (instrucción independiente) ---
    addu $t9, $s1, $t4          # t9 = &Y[i]

    # --- Operación aritmética ---
    mul  $t7, $t6, $t0          # t7 = X[i] * A
    addu $t8, $t7, $t1          # t8 = t7 + B

    # --- Almacenamiento ---
    sw   $t8, 0($t9)            # Y[i] = resultado

    # --- Siguiente iteración ---
    addi $t3, $t3, 1            # i++
    j    loop

fin:
    # --- Finalización ---
```

```
    li   $v0, 10
    syscall
```

2.2. Justificación Técnica de la Mejora

Explica qué instrucción moviste y por qué colocarla entre el `lw` y el `mul` elimina el riesgo de datos:

> En el código base, la instrucción lw $t6, 0($t5) carga desde memoria el valor correspondiente al elemento X[i]. En un pipeline MIPS clásico de cinco etapas (IF, ID, EX, MEM y WB), el dato leído desde memoria no está disponible inmediatamente después de la instrucción de carga, sino hasta la etapa de escritura en el banco de registros (WB). Si la instrucción siguiente utiliza directamente el registro $t6, como ocurre con mul $t7, $t6, $t0, se produce un riesgo de datos tipo RAW (Read After Write), específicamente un hazard Load-Use. En este caso, la multiplicación intenta usar un valor que aún no ha completado su propagación por el pipeline, obligando al procesador a insertar un ciclo de parada (stall) para evitar un resultado incorrecto.

> Para eliminar este riesgo, se aplicó una técnica de reordenamiento de instrucciones (software scheduling) dentro del bucle. Se movió la instrucción addu $t9, $s1, $t4, encargada de calcular la dirección de almacenamiento del resultado Y[i], colocándola inmediatamente después del lw y antes de la instrucción mul. Esta instrucción es independiente del registro $t6, por lo que puede ejecutarse mientras el dato cargado desde memoria termina de avanzar por las etapas del pipeline. De esta manera, el ciclo que antes se desperdiciaba como burbuja se reemplaza por trabajo útil.

> Adicionalmente, se adelantó la instrucción addi $t3, $t3, 1, correspondiente al incremento del índice del bucle. Esta reorganización contribuye a separar dependencias entre instrucciones consecutivas y ayuda a mantener un flujo continuo de ejecución dentro del pipeline sin alterar la lógica del programa. Es importante destacar que el cálculo del desplazamiento y de las direcciones ya se había realizado antes del incremento, por lo que la corrección funcional del algoritmo no se ve afectada.

> Como resultado técnico, la optimización no reduce el número total de instrucciones ejecutadas, pero sí mejora la eficiencia del pipeline al evitar la inserción de ciclos de espera innecesarios. Se logra mantener ocupadas las etapas del procesador con instrucciones útiles, incrementando el aprovechamiento de la segmentación sin modificar el resultado final del programa. Esta mejora demuestra el impacto del orden de las instrucciones en el desempeño de arquitecturas segmentadas.

---

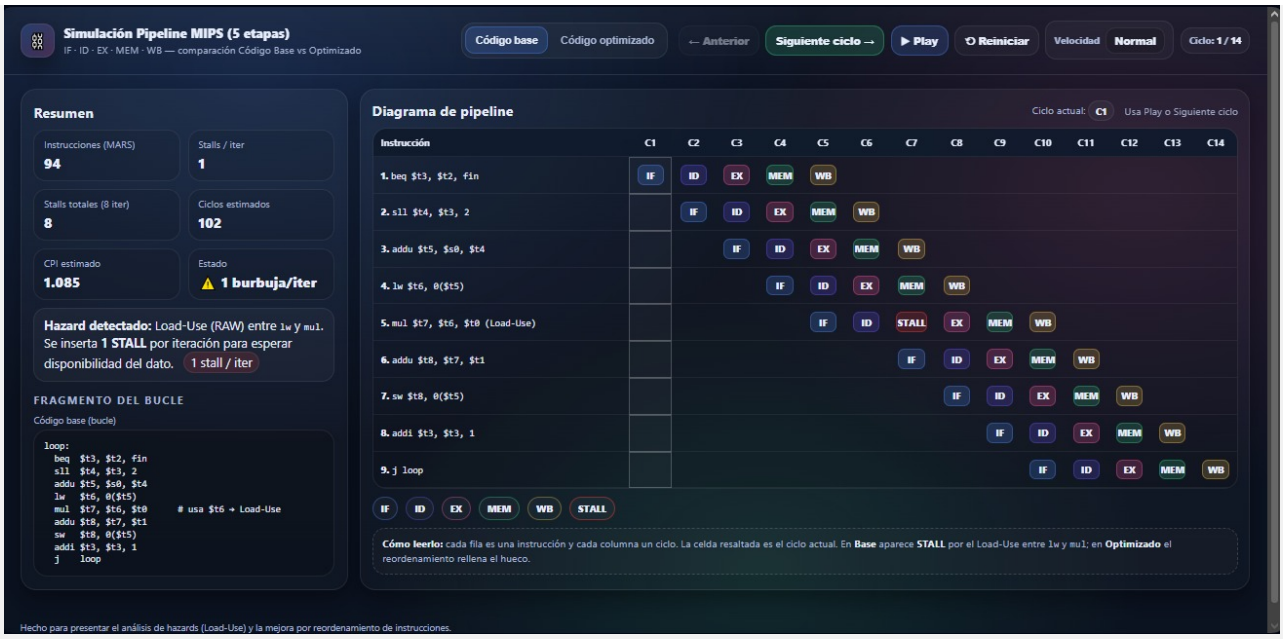
3. Comparativa de Resultados

Métrica	Código Base	Código Optimizado	Mejora (%)
Ciclos Totales	102	94	7,84%
Stalls (Paradas)	1	0	100%
CPI	1,085	1,00	7,83%

Análisis comparativo

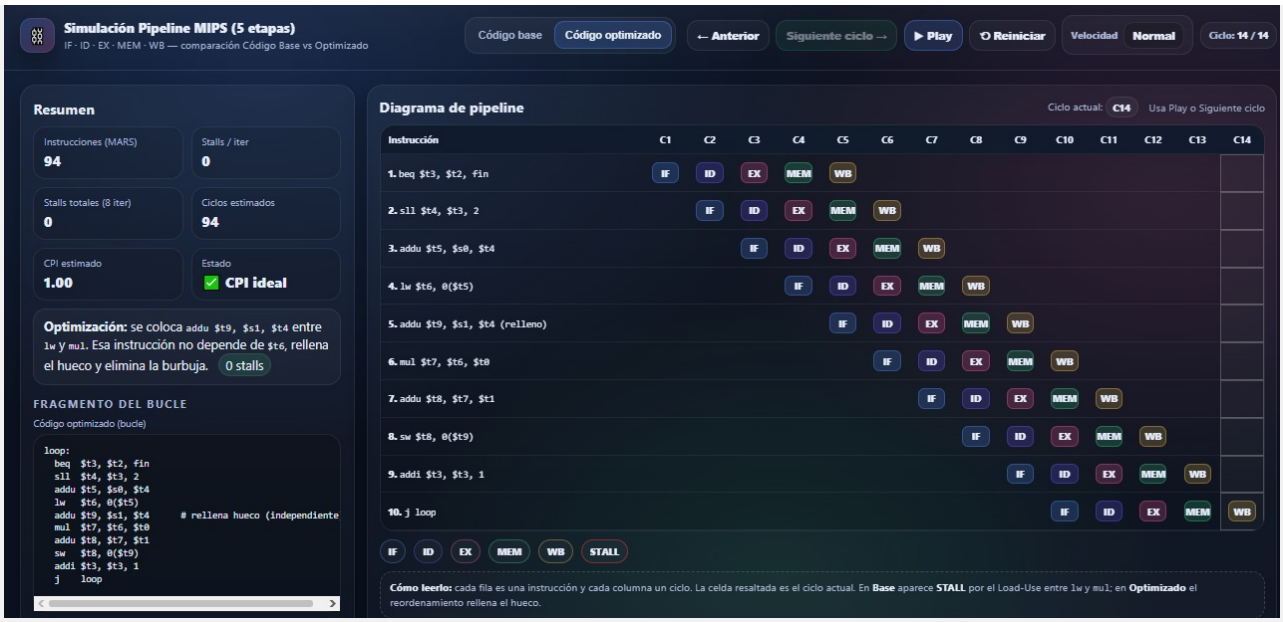
> Aunque MARS reporta el mismo número de instrucciones en ambas versiones, en un procesador segmentado real el tiempo total (ciclos) cambia debido a los stalls causados por hazards. En el código base se estima 1 stall por iteración (8 stalls en total), mientras que el código optimizado rellena el hueco del Load-Use con una instrucción independiente, reduciendo los stalls a 0. Como resultado, el CPI estimado se acerca al ideal (≈1), aumentando el rendimiento.

3.1. Diagrama del Pipeline – Código Base



En el código base se observa un riesgo de datos tipo Load-Use entre las instrucciones `lw` y `mul`. La multiplicación intenta utilizar el valor cargado desde memoria antes de que esté disponible en el pipeline, lo que obliga al procesador a insertar una burbuja (STALL). Esta parada incrementa los ciclos totales de ejecución y eleva el CPI estimado.

3.2. Diagrama del Pipeline – Código Optimizado



En el código optimizado se reorganizaron las instrucciones para insertar una operación independiente entre `lw` y `mul`. Este reordenamiento elimina el riesgo Load-Use, evitando la inserción de burbujas en el

> pipeline. Como resultado, se mantiene la continuidad del flujo de ejecución, se reducen los ciclos totales y el CPI se aproxima a 1.0.

4. Conclusiones

¿Qué impacto tiene la segmentación en el diseño de software de bajo nivel? ¿Es siempre posible eliminar todas las paradas?

> El análisis realizado demuestra que la segmentación (pipeline) no solo es un mecanismo de aceleración a nivel hardware, sino que influye directamente en la forma en que debe diseñarse el software de bajo nivel. En arquitecturas segmentadas, el orden de las instrucciones impacta el rendimiento tanto como la cantidad de instrucciones ejecutadas. La optimización implementada evidenció que una adecuada reorganización del código permite reducir riesgos de datos sin alterar la funcionalidad del programa, mejorando el aprovechamiento de las etapas del pipeline.

> En particular, la eliminación del hazard tipo Load-Use mediante reordenamiento manual de instrucciones confirma que el rendimiento no depende exclusivamente del hardware, sino también de la forma en que el programador (o compilador) estructura el flujo de ejecución. Al sustituir un ciclo de espera por una instrucción independiente, se logró mantener el pipeline ocupado y reducir el CPI, demostrando que pequeñas modificaciones en el orden del código pueden generar mejoras medibles en la eficiencia.

> Sin embargo, no siempre es posible eliminar completamente todas las paradas. Existen dependencias de datos inevitables cuando una instrucción requiere necesariamente el resultado inmediato de otra, así como riesgos de control asociados a saltos condicionales cuya resolución depende del flujo del programa. Además, ciertas latencias están determinadas por el propio diseño del hardware y no pueden ser eliminadas únicamente mediante optimización de software.

> En conclusión, la segmentación obliga a pensar el software no solo en términos funcionales, sino también temporales. El programador debe considerar cuándo los datos estarán disponibles y cómo organizar las instrucciones para alimentar continuamente el pipeline. La optimización aplicada en este laboratorio evidencia cómo el conocimiento de la arquitectura permite escribir código más eficiente, reforzando la estrecha relación entre diseño de hardware y programación de bajo nivel.